

RAPPORT DE STAGE : MISSION ASSISTANT INGÉNIEUR

Conception, Benchmark et Déploiement d'un Simulateur Vocal d'Intelligence Artificielle pour l'Entraînement des Conseillers Bancaires

Organisme d'accueil : BMCI (Banque Marocaine pour le Commerce et l'Industrie)

Institution : École Centrale Casablanca (ECC)

Auteur : Élève Ingénieur de Centrale Casablanca

Période de stage : 2024

Sommaire

1. Introduction et Formulation du Problème

- 1.1. Contexte industriel (BMCI)
- 1.2. Problématique et besoins de formation
- 1.3. Enjeux opérationnels et objectifs

2. Cartographie et Étude Comparative de l'État de l'Art

- 2.1. Cartographie des modèles ASR/STT
- 2.2. Leaderboard des modèles de langage (LLM) en français
- 2.3. Étude comparative des modèles de synthèse vocale (TTS)

3. Architecture de la Solution Temps Réel (LiveKit & WebRTC)

- 3.1. Choix technologique de LiveKit
- 3.2. Architecture de la pipeline voix-à-voix
- 3.3. Analyse des fonctionnalités de dialogue naturel (Full-Duplex, Barge-in)

4. R&D et Résolution des Verrous Techniques

- 4.1. Résolution du crash WebRTC par rééchantillonnage dynamique (24kHz à 48kHz)
- 4.2. Conception du superviseur anti-crash et de résilience (`run_agent.py`)
- 4.3. Implémentation du mécanisme de secours (fallback) STT
- 4.4. Ingénierie de prompt pour la modération et l'identification de la cliente (Mme. Sarah Bennani)

5. Résultats Obtenus et Perspectives d'Industrialisation

- 5.1. Déploiement du scénario final
- 5.2. Limites de l'architecture actuelle
- 5.3. Prolongements : RAG marocain et classification émotionnelle acoustique

6. Bibliographie

1. Introduction et Formulation du Problème

1.1. Contexte industriel (BMCI)

La Banque Marocaine pour le Commerce et l'Industrie (BMCI), filiale du groupe BNP Paribas, est un acteur majeur du secteur bancaire marocain. Face à la digitalisation rapide des services financiers et à la diversification des attentes clients, la qualité de la relation client reste un pilier de différenciation stratégique. Les centres de relation client et les conseillers en agence sont en première ligne pour gérer des situations complexes, parfois conflictuelles.

1.2. Problématique et besoins de formation

Pour maintenir un niveau d'excellence de service, les conseillers doivent être entraînés de manière continue à la gestion des clients mécontents, stressés ou indécis. Traditionnellement, cet entraînement repose sur des jeux de rôle (roleplay) animés par des formateurs humains. Cependant, cette méthode présente des limites majeures :

- **Coût élevé** et faible scalabilité (nécessite la présence constante d'un formateur).
- **Subjectivité** de l'évaluation du formateur.
- **Manque de répétabilité** pour un même apprenant dans des conditions rigoureusement identiques.

Le but de cette mission assistant ingénieur est de concevoir et de déployer un **simulateur vocal basé sur l'intelligence artificielle** capable d'incarner de façon autonome, réaliste et dynamique des profils de clients types lors d'un appel téléphonique simulé.

1.3. Enjeux opérationnels et objectifs

Pour que cette simulation soit pédagogiquement efficace, elle doit répondre à plusieurs exigences techniques strictes :

1. **Fluidité conversationnelle (Latence < 1s)** : Les interlocuteurs téléphoniques ne tolèrent pas de blanc ou de pause artificielle. Le système doit écouter, réfléchir et répondre en moins d'une seconde.
2. **Expressivité vocale (TTS réaliste)** : L'IA doit être capable d'exprimer des émotions (irritation, colère, stress, soulagement) de manière naturelle.
3. **Robustesse et résilience** : La solution doit tolérer les bruits de fond des PC des apprenants, les coupures réseau et les surcharges d'API (erreurs HTTP 429).
4. **Pertinence du scénario** : Le comportement du client simulé doit évoluer dynamiquement selon la posture (empathie, rigidité) adoptée par l'apprenant.

2. Cartographie et Étude Comparative de l'État de l'Art

Avant d'entamer le développement de l'agent, une étude cartographique approfondie a été menée pour répertorier et évaluer les meilleurs modèles de reconnaissance vocale (ASR/STT), de traitement du langage (LLM) et de synthèse vocale (TTS) disponibles.

2.1. Cartographie des modèles ASR/STT

L'évaluation s'est basée sur les jeux de données francophones classiques (CoVoST, MLS,

Fleurs) en mesurant le **WER** (Word Error Rate - taux d'erreur sur les mots, plus bas = meilleur) et le **RTFx** (Real-Time Factor - vitesse de traitement relative, plus élevé = plus rapide).

Modèle STT / ASR	WER Moyen (%)	Vitesse (RTFx)	Type de modèle	Informations Clés
elevenlabs/scribe_v2	2.67 %	NA	API Propriétaire	Très haute fidélité
CohereLabs/cohere-transcribe-03-2026	3.83 %	491.36	API Propriétaire	Rapide, robuste aux accents
mistralai/Voxtral-Small-24B	3.70 %	42.04	Open-weights	Modèle de 24B paramètres
reson8/resonant-1	3.52 %	NA	API	Disponible en ligne
nvidia/canary-1b-v2	4.60 %	634.37	Open-source	1B params, excellente vitesse
openai/whisper-large-v3	4.81 %	110.92	Open-source	2B params, référence du marché
Qwen/Qwen3-ASR-1.7B	5.11 %	112.77	Open-source	1.7B params, gestion d'accents
openai/whisper-large-v3-turbo	5.56 %	176.16	Open-source	Version compressée (0.8B params)

Impact de l'appareil d'enregistrement sur le WER (Tests réels BMCI)

Des tests d'enregistrement de 3 minutes ont été menés sur trois dispositifs physiques différents pour évaluer l'impact acoustique sur la transcription :

- **PC BMCI de bureau** : WER plus élevé (~8.32% avec Cohere) en raison de l'écho de la pièce de formation et du bruit de la ventilation.
- **Casque audio (Headset)** : Bon rapport signal/bruit (~8.52% avec Cohere), mais signal parfois étouffé.
- **Téléphone portable** : Meilleure clarté vocale du fait de la proximité du micro et de sa réduction de bruit active intégrée (WER à ~5.82% avec Cohere et ~3.12% avec ElevenLabs).

2.2. Leaderboard des modèles de langage (LLM) en français

Les modèles de langage ont été évalués sur le leaderboard de l'IA de l'administration française (coordination-ia) selon leur fidélité de suivi d'instructions en français (IFEval FR), leur raisonnement de haut niveau (GPQA FR) et leur performance générale.

Rang	Modèle LLM	IFEval Fr (%)	GPQA FR (%)	Bac FR (%)	Note globale
1	DeepSeek-R1-Distill-Llama-70B	66.17 %	50.92 %	50.71 %	55.93 %
2	Mistral-Large-Instruct-2411	67.44 %	30.65 %	50.14 %	49.41 %
3	Llama-3.3-70B-Instruct	74.51 %	25.76 %	45.34 %	48.54 %
4	DeepSeek-R1-Distill-Qwen-32B	60.14 %	39.19 %	45.62 %	48.32 %

5	Qwen2.5-72B-Instruct	71.10 %	24.79 %	44.92 %	46.93 %
8	Chocolatine-2-14B-v2.0	52.69 %	22.34 %	51.69 %	42.24 %
10	Qwen2.5-14B-Instruct	66.42 %	15.51 %	43.64 %	41.86 %

Analyse des modèles locaux légers (SLM) : Bien que les modèles de plus de 70B paramètres (Llama-3.3, DeepSeek-R1-Llama) affichent d'excellents résultats, leur déploiement sur les infrastructures locales de la banque sans GPU puissant est impossible. L'évaluation de modèles plus légers (comme Qwen-14B ou Chocolatine-14B) a montré un bon compromis pour une future intégration locale.

2.3. Étude comparative des modèles de synthèse vocale (TTS)

Pour reproduire la colère, les modèles de synthèse vocale ont été testés sur leur expressivité émotionnelle (note MOS de 1 à 5) et leur latence de réponse :

Rang	Modèle TTS	Type	MOS (1-5)	Latence moyenne	RTF
	☐ Hume AI (Octave)	API	4.03	1.97 s	0.535
	☐ F5-TTS	Local	3.79	127.91 s (sur CPU)	37.435
	☐ Mistral Voxtral	API	3.78	1.57 s	0.363
#4	Edge-TTS	API	3.61	0.70 s	0.124
#5	Kokoro v0.19	Local	3.56	3.03 s	0.700
#7	ElevenLabs	API	3.45	1.96 s	0.526

Choix technologique final : Bien que Hume AI offre un rendu de voix très naturel, sa latence moyenne de près de 2 secondes brise le rythme d'une conversation téléphonique. **Mistral Voxtral (avec la voix fr_marie_angry)** a été choisi pour sa latence réduite (1.57s) et sa capacité native à exprimer de manière convaincante une irritation et une impatience sans nécessiter de post-traitement.

3. Architecture de la Solution Temps Réel (LiveKit & WebRTC)

3.1. Choix technologique de LiveKit

Pour construire une pipeline interactive de voix à voix sans effet "talkie-walkie", le protocole HTTP standard ou les connexions WebSocket brutes s'avèrent insuffisants. Nous avons sélectionné **LiveKit**, un framework open-source basé sur le protocole **WebRTC** (utilisé par Discord et Zoom). Les avantages de WebRTC pour ce projet sont :

- **Streaming audio ultra-rapide :** L'audio est envoyé sous forme de paquets RTP continus.
- **Gestion automatique de la gigue (jitter)** et de la perte de paquets.
- **Écho-annulation intégrée (AEC)** côté client.

3.2. Architecture de la pipeline voix-à-voix

L'agent vocal est configuré comme un participant virtuel dans le salon LiveKit, s'abonnant au flux du micro de l'apprenant et publiant son propre flux audio de réponse. La pipeline

s'organise en trois briques séquentielles et asynchrones :

```
[Voix Conseiller] → [WebRTC Audio Stream] → [STT (Cohere / Whisper)] →  
|  
[Audio Réponse] → [TTS (Mistral Voxtral)] → [LLM (Mistral Small)] → [
```

3.3. Analyse des fonctionnalités de dialogue naturel

Pour reproduire le comportement d'un vrai appel téléphonique, la pipeline tire parti des fonctionnalités avancées de LiveKit :

- **Full-Duplex** : L'IA et le conseiller peuvent parler simultanément.
- **Barge-in (Interruption immédiate)** : Si le client IA est en train de s'énervier et de parler, et que le conseiller l'interrompt pour proposer une solution, l'agent LiveKit coupe immédiatement son émission audio pour écouter la nouvelle réplique de l'apprenant.
- **Réglage du délai d'endpointing (`min_delay=0.8s`)** : L'agent attend une pause de silence d'au moins 0,8 seconde avant de valider la fin de la réplique du conseiller, évitant de lui couper la parole au milieu d'une phrase hésitante.

4. R&D et Résolution des Verrous Techniques

Durant le développement, plusieurs verrous techniques complexes liés à l'environnement d'exécution Windows et à la résilience des API ont été résolus.

4.1. Résolution du crash WebRTC par rééchantillonnage dynamique

- **Le problème** : La synthèse vocale de Mistral Voxtral sort un flux audio échantillonné à **24kHz**. Cependant, lors de l'injection de ces données dans le canal de communication asynchrone de LiveKit sous Windows, la bibliothèque native écrite en Rust (`webrtc-sys`) paniquait systématiquement en tentant de décoder des paquets non standardisés, provoquant le crash immédiat de tout le processus Python.
- **La solution** : Nous avons conçu et inséré un module de traitement audio intermédiaire dans le script `[agent.py]` (`file:///C:/Users/user/.gemini/antigravity/scratch/tts-benchmark/agent.py`) pour intercepter le flux brut de Mistral, et le rééchantillonner en **48kHz** (le standard natif de WebRTC) à l'aide de la classe `rtc.AudioResampler` de LiveKit avant son émission.

```
# Extrait de code de la solution de rééchantillonnage implémentée  
resampler = rtc.AudioResampler(  
    input_sample_rate=24000,  
    output_sample_rate=48000,  
    input_channels=1,  
    output_channels=1  
)  
# Les paquets de 24kHz sont injectés et convertis de manière fluide à 1
```

4.2. Conception du superviseur anti-crash et de résilience (`run_agent.py`)

- **Le problème** : WebRTC gère de nombreux threads parallèles pour le flux audio. Lors

d'une déconnexion d'utilisateur (par exemple, si l'apprenant ferme son navigateur), la libération asynchrone des ressources réseaux par Rust provoquait un blocage (deadlock) ou un crash avec un code de retour Windows non nul. Cela obligeait l'administrateur à relancer manuellement le worker en console.

- **La solution** : Développement d'un script de supervision résilient [run_agent.py](file:///C:/Users/user/.gemini/antigravity/scratch/tts-benchmark/run_agent.py). Ce script lance l'agent dans un sous-processus isolé. Il écoute en continu les signaux de sortie et, en cas de crash, de panic Rust ou de déconnexion anormale, il tue proprement les sockets et **recrée un nouveau worker propre en moins de 2 secondes**.

4.3. Implémentation du mécanisme de secours (fallback) STT

- **Le problème** : L'API de transcription de Cohere Transcribe v2, bien que très précise sur les accents bancaires, est limitée par un quota de requêtes par minute (Rate Limit) strict. Le bruit de fond persistant sur les micros des ordinateurs de bureau déclenchait le VAD en boucle, générant de nombreuses requêtes de silence qui saturaient le quota de Cohere en moins de 2 minutes (Erreurs HTTP 429), rendant l'agent muet.
- **La solution** : Refactoring de la classe `CohereSTT` pour intercepter les exceptions réseau et les codes de retour 429. En cas d'échec de Cohere, l'agent bascule **instantanément et de manière transparente** sur l'API de secours **OpenAI Whisper STT** pour traiter la réplique en cours de discussion, garantissant une continuité absolue de l'expérience utilisateur.

4.4. Ingénierie de prompt pour la modération et l'identification de la cliente (Mme. Sarah Bennani)

Suite aux retours de la présentation devant les responsables de la BMCI, des ajustements majeurs ont été apportés au prompt de l'agent :

- **Interdiction des indications de jeu (* . . *)** : Le modèle de langage avait tendance à inclure des didascalies théâtrales dans son texte (comme **Frappe sur le comptoir** ou **Soupir exaspéré**). Non seulement cela parasitait l'interface de chat, mais la synthèse vocale lisait parfois ces expressions à haute voix. Une consigne stricte a été ajoutée pour prohiber toute description physique.
- **Identification progressive** : La cliente fictive (Mme. Sarah Bennani) a pour consigne de ne pas divulguer son nom, son numéro de CIN (AB123456) ou sa dernière transaction (un dépôt de 15 000 DH la semaine dernière) **que si le conseiller lui formule une demande polie et réglementaire d'identification**. Cela force le stagiaire à respecter les procédures de sécurité bancaire standard en matière de KYC (Know Your Customer).
- **Humeur réactive** : L'agent adapte son agacement. Si le conseiller est calme et empathique, l'IA se détend progressivement. S'il est froid ou rigide, elle exige de parler au directeur de l'agence.

5. Résultats Obtenus et Perspectives d'Industrialisation

5.1. Déploiement du scénario final

L'agent vocal est déployé et opérationnel sur l'infrastructure bac à sable LiveKit Cloud à l'adresse de connexion : `wss://internship-obt2eynj.livekit.cloud`

Le scénario de **Mme. Sarah Bennani** permet de tester concrètement :

1. **La réactivité** : Réponse générée en moins de 1,2 seconde.
2. **Le respect des règles de sécurité** : L'apprenant doit obligatoirement valider l'identité de Mme. Bennani avant de lui proposer des solutions de retrait exceptionnel de ses 100 000 DH (comme un virement immédiat).
3. **Le format téléphonique** : La salutation d'entrée simule un appel direct et non un entretien physique.

5.2. Limites de l'architecture actuelle

- **Dépendance aux connexions Internet (API Cloud)** : L'utilisation d'API cloud (Mistral, Cohere, OpenAI) rend la latence globale dépendante de la bande passante du réseau internet de la BMCI.
- **Facturation à l'usage** : Une utilisation massive de la plateforme par des dizaines de stagiaires simultanément générera un coût récurrent d'appels d'API.

5.3. Prolongements et Perspectives d'Industrialisation

Pour l'industrialisation finale au sein de la banque, deux pistes majeures sont à privilégier :

- **A. Intégration d'un RAG (Retrieval-Augmented Generation)** : Connecter une base de connaissances vectorielle contenant la documentation interne de la BMCI (adresses réelles des agences, grilles tarifaires de tenue de compte, conditions d'octroi de cartes) pour que l'IA puisse citer des informations bancaires réelles lors de la conversation.
- **B. Détection émotionnelle par classification acoustique** : Développer un modèle de classification audio léger en temps réel (ex: Wav2Vec2) pour détecter si la voix humaine du stagiaire est calme, stressée ou agressive, et envoyer ce score émotionnel comme métadonnée au LLM pour moduler l'irritation de la cliente de manière encore plus réaliste.

6. Bibliographie

1. **LiveKit Documentation** - *Real-time audio and WebRTC streaming for AI agents*, <https://docs.livekit.io>
2. **Mistral AI Developer Platform** - *Voxtral Text-to-Speech models and API specifications*, <https://docs.mistral.ai>
3. **Cohere API Reference** - *Cohere Transcribe v2 model benchmarks and capabilities*, <https://docs.cohere.com>
4. **OpenAI Whisper** - *Robust Speech Recognition via Large-Scale Weak Supervision*, Radford et al., 2022.
5. **Hugging Face Open ASR Leaderboard** - *Benchmarks and Word Error Rates on French datasets*, https://huggingface.co/spaces/hf-audio/open_asr_leaderboard